NEW EMPLOYEE ONBOARDING | ARCHITECTURE & SECURITY MANUAL

SecurePortal: Enterprise Full-Stack Reference Application

Complete Master Prompt, System Architecture, 15 Security Pillars Deep-Dive, and Step-by-Step Local Machine Setup Guide for New Engineering Hires

Attribute	Specification / Value
Project Location	C:\Projects\secure-portal
Frontend Stack	Angular 22 (Zoneless Change Detection, Signals, Standalone Components with separated .ts/.html/.css files)
Backend Stack	.NET 10 Web API (C# 14, Clean Architecture: Domain, Application, Infrastructure, Api)
Database & Caching	SQL Server (SecurePortalDb on localhost:1433) + EF Core 10 + .NET 10 HybridCache
Security & Audit Status	15 Security Pillars Enforced | 22/22 E2E Security Tests Passing | 0 NuGet/NPM Vulnerabilities
Default Admin Login	Username: admin (or admin@secureportal.local) | Password: Admin@Secure2026!P@ss

SECTION 1

Executive Overview — What SecurePortal Does

SecurePortal is an enterprise-grade, highly performant **Secure Document & Task Management Portal**. It was architected from the ground up to serve as our team's gold-standard reference implementation across 15 mandatory application security pillars and modern full-stack performance patterns.

1.1 Five Core Functional Modules

- **1. Identity, Authentication & Role-Based Access Control (RBAC):** Supports user registration and sign-in using dual-mode authentication: JWT Bearer tokens paired with HttpOnly, SameSite=Strict secure cookies and Refresh Token Rotation (RTR). Enforces three roles: Admin, Manager, and Member via ASP.NET authorization policies and Angular functional guards (authGuard, adminGuard).
- **2. Interactive Security Dashboard & Telemetry:** Displays live workflow counters (Total Tasks, In Progress, Completed, and SHA-256 Verified Documents). Uses Angular 22 @defer (on viewport) to lazily render the Security & Runtime Telemetry widget (Session Protection, SQL Data Protection key status, and Sliding Window Rate Limiter status).
- **3. High-Performance Task Management:** Provides full task CRUD, search, and status/priority filtering. Implements **Keyset Cursor Pagination** (base64(CreatedAtUtc|Id)) instead of SQL OFFSET/FETCH for constant O(1) database read performance, backed by .NET 10 HybridCache with tag-based cache eviction (tasks tag).
- **4. Cryptographic Documents Vault:** Drag-and-drop file upload vault enforcing a 25 MB limit and blocking dangerous executable/script extensions (.exe, .dll, .bat, .cmd, .ps1, .sh, .msi). Computes a cryptographic **SHA-256 hash** for every uploaded document to verify integrity.
- **5. Admin Compliance Audit Trail & JSON Diff Inspector:** Restricted strictly to Admin users (/api/audit). Powered by AuditableEntityInterceptor, which automatically captures every database insert, update, and soft-delete, recording old/new JSON values while masking sensitive fields (PasswordHash, RefreshToken, SecurityStamp) as [REDACTED].

1.2 End-to-End Request Lifecycle

- **Step 1 (CSRF Bootstrap):** On startup (App.ngOnInit), Angular calls GET /api/antiforgery/token with withCredentials: true, seeding the browser with a strict XSRF-TOKEN cookie.
- **Step 2 (Client Interceptors):** Outgoing requests pass through authInterceptor (attaching Authorization: Bearer <token> and withCredentials: true) and Angular's XSRF interceptor (copying XSRF-TOKEN to the X-XSRF-TOKEN header).

- **Step 3 (API Middleware Pipeline):** Requests traverse SecurityHeadersMiddleware -> GlobalExceptionHandler (RFC 7807) -> UseCors -> UseRateLimiter -> UseAuthentication -> UseAuthorization -> UseAntiforgery.
- **Step 4 (Persistence & Audit):** Validated DTOs query SQL Server via parameterized EF Core 10 LINQ queries and HybridCache, while AuditableEntityInterceptor logs redacted audit records.

SECTION 2

The Complete Original Master Prompt (Verbatim)

Below is the exact, unabridged Master Prompt used to design and build SecurePortal across all five phases:

ROLE & OBJECTIVE

You are a Principal Software Architect and Application Security (AppSec) Specialist. Your objective is to design and implement a production-grade, highly performant, end-to-end sample application using:

- Frontend: Angular 22 (Standalone Components, Signals, Zoneless, Modern Control Flow)

- Backend: .NET 10 Web API (C# 14, Clean/Vertical Slice Architecture, EF Core 10)

- Database: Microsoft SQL Server (Docker-ready, EF Core Migrations, Indexing best practices)

1. Authentication & Authorization

- Implement token-based authentication using the Backend-For-Frontend (BFF) pattern or Secure Cookie-backed JWT with Refresh Token Rotation.

- In Angular: Functional guards (canActivate: [authGuard]), functional role/permission resolvers, and state stored in Signals.

- In .NET: Policy-based and Claims-based authorization ([Authorize(Policy = '...')]), fine-grained permission handlers (IAuthorizationRequirement).

2. Transport Layer Security (HTTPS & HSTS)

- Force HTTPS redirection with status code 308/301.

- Configure HTTP Strict Transport Security (HSTS) with max-age=31536000; includeSubDomains; preload.

- Modern TLS cipher suites and TLS 1.3 enforcement.

3. Input Validation

- Backend: FluentValidation auto-wired into the ASP.NET pipeline with automatic standard RFC 7807 ProblemDetails validation responses. Zero tolerance for unvalidated DTOs.

- Frontend: Typed Reactive Forms (FormGroup, FormControl), custom validators, and sanitation before sending payloads.

4. SQL Injection Prevention

- Enforce EF Core LINQ parameterized queries by default.

- Never use dynamic string concatenation or raw string interpolation in SQL queries.

- If raw SQL is needed, use FromSqlInterpolated or parameterized Dapper queries with explicit DbParameter mapping.

- Apply database principle of least privilege (app user cannot execute DDL or drop tables).

5. Cross-Origin Resource Sharing (CORS)

- Strict origin whitelisting loaded from environment/configuration (no wildcards * when credentials are permitted).

- Explicit allowed headers (Content-Type, Authorization, X-XSRF-TOKEN), exposed headers, and explicit methods (GET, POST, PUT, DELETE).

6. Cross-Site Request Forgery (CSRF / XSRF)

- Implement Double-Submit Cookie pattern or ASP.NET Core IAntiforgery token validation.

- Angular functional HTTP interceptor (xsrfInterceptor) reading the token from XSRF-TOKEN cookie and setting the X-XSRF-TOKEN request header.

- Antiforgery middleware applied to all state-changing endpoints (POST, PUT, DELETE, PATCH).

7. Rate Limiting & DoS Protection

- Use .NET 10 built-in Microsoft.AspNetCore.RateLimiting.

- Configure sliding window and token bucket policies by client IP and authenticated User ID.

- Dedicated stricter rate-limiting on sensitive endpoints (/api/auth/login, /api/auth/register, /api/auth/refresh-token).

- Return HTTP 429 Too Many Requests with standard Retry-After headers.

8. Secure Cookie Architecture

- Authentication and session cookies must have flags: HttpOnly = true (inaccessible to JavaScript), Secure = true (transmitted only over HTTPS), SameSite = SameSiteMode.Strict, and cookie naming prefixed with __Host- or __Secure- (e.g., __Host-access-token).

9. Secrets Management & Configuration

- Zero secrets, connection strings, or credentials committed to source code.
- Local Development: Use .NET dotnet user-secrets and Angular environment substitution.
- Production: Environment variables, Azure Key Vault, or Docker Secret mounts.
- Implement options validation on startup using .ValidateDataAnnotations().ValidateOnStart().

10. Security Response Headers

- Inject hardened HTTP security headers via middleware: Content-Security-Policy (default strict CSP prohibiting unsafe-eval and framing), X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy (disable camera, microphone, geolocation, payment), and remove server banner headers (X-Powered-By, Server: Kestrel).

11. Error Handling & RFC 7807 Problem Details

- Backend: Centralized error handling using .NET 10 IExceptionHandler. Map domain exceptions to RFC 7807 ProblemDetails with unique trace IDs (traceparent / Activity.Current?.Id). No stack traces or internal DB details leaked in production.
- Frontend: Global ErrorHandler and functional HttpInterceptor to catch HTTP 400, 401, 403, 404, 429, 500 and display user-friendly toast/modal notifications.

12. Secure Logging & Auditing

- Structured logging using Serilog or OpenTelemetry with JSON output.
- Log enrichment with CorrelationId, UserId, ClientIP.
- Automatic PII/Secret masking/redaction (strip passwords, tokens, credit card numbers, SSNs from logs).
- Dedicated audit log entries for authentication attempts, permission failures, and critical data modifications.

13. Data Protection & Cryptography

- Configure ASP.NET Core Data Protection API (AddDataProtection()) with persistent key storage and encryption keys at rest.
- Passwords hashed using standard PBKDF2 / Argon2id via ASP.NET Identity Core IPasswordHasher<T>.
- Sensitive database fields encrypted at rest using EF Core Value Converters or SQL Server Always Encrypted.

14. Dependency Security & Supply Chain

- package.json with strict versioning and package-lock.json checked into Git.
- Directory.Packages.props for Central Package Management (CPM) in .NET with vulnerability auditing enabled (<NuGetAudit>true</NuGetAudit>).
- Multi-stage Dockerfiles running as non-root user (USER app / node).

15. High Performance Engineering

- Backend: Leverage .NET 10 HybridCache for L1/L2 multi-tier caching, EF Core .AsNoTracking() on read queries, Keyset (cursor) pagination, and System.Text.Json source generators.
- Frontend: Angular 22 Signals (signal, computed, effect), Zoneless mode enabled (provideZonelessChangeDetection()), Deferrable views (@defer (on viewport)), Modern control flow (@if, @for, @switch), and Route lazy loading.

5-PHASE IMPLEMENTATION PLAN

- Phase 1: Project Setup & Infrastructure (docker-compose.yml, .NET 10 Clean Architecture solution, Angular 22 workspace)
- Phase 2: Database Layer & Data Protection (EF Core 10 DbContext, Fluent mappings, AuditableEntityInterceptor, DataProtectionKeys, initial migration & seeding)

- Phase 3: .NET 10 Security Engine & API Pipeline (SecurityHeadersMiddleware, GlobalExceptionHandler, RateLimiting, Antiforgery, Auth/Tasks/Documents/Audit controllers)
- Phase 4: Angular 22 Frontend Implementation (Zoneless config, Signals state, functional interceptors/guards, standalone views with separated .ts, .html, .css files)
- Phase 5: Verification & Security Testing Checklist (22-point automated PowerShell verification suite + dependency vulnerability audits)

SECTION 3

Clean Architecture & Repository Blueprint

The repository at C:\Projects\secure-portal follows strict Clean Architecture on the backend and a modular, feature-based standalone architecture on the Angular 22 frontend:

```
C:\Projects\secure-portal\
+-- backend/
|   +-- Directory.Build.props               # C# 14, Nullable=enable, AnalysisLevel=latest
|   +-- Directory.Packages.props            # Central Package Management (CPM) & NuGetAudit=true
|   +-- nuget.config                        # Isolated official NuGet feed
|   +-- SecurePortal.slnx                   # .NET 10 XML Solution (4 Projects)
|   \-- src/
|       +-- Domain/                         # Entities (User, Role, TaskItem, Document, AuditEntry)
|       +-- Application/                    # DTOs, FluentValidation Validators, Keyset PagedResult
|       +-- Infrastructure/                 # EF Core 10 AppDbContext, AuditableEntityInterceptor, Seed
|       \-- Api/                            # Controllers, SecurityHeadersMiddleware, GlobalExceptionHandler
+-- frontend/
|   +-- angular.json                        # Build/Serve configs with development fileReplacements
|   +-- package.json                        # Angular 22 standalone dependencies (0 vulnerabilities)
|   \-- src/
|       +-- environments/
|       |   +-- environment.ts              # Production config (production: true, apiUrl: '/api')
|       |   \-- environment.development.ts  # Development config (apiUrl: 'http://localhost:5047/api')
|       \-- app/
|           +-- app.ts / app.html / app.css  # Root component (seeds XSRF-TOKEN cookie on init)
|           +-- app.config.ts               # provideZonelessChangeDetection(), functional interceptors
|           +-- app.routes.ts               # Lazy-loaded routes protected by authGuard & adminGuard
|           +-- core/                       # Services, Guards, Interceptors (auth, error), Models
|           +-- shared/components/          # Separated .ts, .html, .css for Navbar & Toast
|           \-- features/                   # Separated .ts, .html, .css for Login, Register,
|                                           # Dashboard, Tasks, Documents, and Audit
\-- scripts/
    +-- setup.ps1                           # Full project initialization script
    +-- dev-certs.ps1                       # Local HTTPS certificate trust script
    +-- sql-init.sql                        # SQL Server snapshot isolation setup script
    +-- start-dev.ps1                       # One-command concurrent Backend + Frontend launcher
    +-- test-api.ps1                        # Core API verification script
    \-- test-security-e2e.ps1               # 22-check automated security & performance suite
```

3.2 SQL Server Database Tables (SecurePortalDb)

Table Name	Key Columns / Indexes	Purpose & Security Controls
Users	Id, Username, Email, PasswordHash	Stores PBKDF2 password hashes, security stamps, and refresh tokens. Soft-delete filter applied.
Roles & UserRoles	Id, Name, UserId + RoleId	Many-to-many RBAC mapping seeded with Admin, Manager, and Member roles.
TaskItems	IX_TaskItems_CreatedAtUtc_Id	Composite index on (CreatedAtUtc DESC, Id DESC) enabling O(1) keyset cursor pagination.
Documents	Id, HashSha256, StoragePath	Stores file metadata, MIME content type, byte size, and 64-char SHA-256 cryptographic checksum.
AuditEntries	Id, TimestampUtc, EntityName	Immutable compliance log storing JSON diffs with sensitive fields masked as [REDACTED].
DataProtectionKeys	Id, FriendlyName, Xml	Persists ASP.NET Core Data Protection key rings in SQL Server across server restarts.

SECTION 4

Deep-Dive into the 15 Mandatory Security Pillars

#	Security Pillar	Implementation Mechanism	Key Files
01	**Authentication**	Dual-mode JWT Bearer + HttpOnly Cookie auth with Refresh Token Rotation (RTR).	AuthController.cs, TokenService.cs
02	**Authorization (RBAC)**	Role policies (Admin, Manager, Member) + Angular functional guards.	AuditController.cs, admin.guard.ts
03	**HTTPS & HSTS**	Strict HSTS in production + HTTPS redirection + dev certificate trust script.	Program.cs, dev-certs.ps1
04	**Input Validation**	FluentValidation rules enforcing strong passwords, email format, and field lengths.	AuthValidators.cs, TaskValidators.cs
05	**SQL Injection**	100% parameterized EF Core 10 LINQ queries; verified against SQLi payloads.	AppDbContext.cs, TasksController.cs
06	**CORS**	Whitelists http://localhost:4200 & https://localhost:4200 with credentials.	Program.cs, appsettings.json
07	**CSRF / Antiforgery**	Double-submit XSRF-TOKEN cookie paired with X-XSRF-TOKEN header.	AntiforgeryController.cs, app.config.ts
08	**Rate Limiting**	Sliding window global (100/min) + Fixed window auth (10/min) returning HTTP 429.	Program.cs, AuthController.cs
09	**Secure Cookies**	HttpOnly=true, SameSite=Strict, and dynamic __Host- prefix on HTTPS.	AuthController.cs
10	**Secrets Management**	.env isolation, User Secrets support, and startup configuration checks.	.env.example, Program.cs
11	**Security Headers**	Strict CSP, X-Frame-Options: DENY, nosniff, and Kestrel Server strip.	SecurityHeadersMiddleware.cs
12	**Error Handling**	RFC 7807 ProblemDetails via IExceptionHandler with traceId correlation.	GlobalExceptionHandler.cs
13	**Secure Logging**	Serilog PII Masking + Audit Interceptor redacting PasswordHash & RefreshToken.	AuditableEntityInterceptor.cs
14	**Data Protection**	ASP.NET Data Protection keys persisted to SQL Server DataProtectionKeys table.	DependencyInjection.cs
15	**Dependency Security**	Central Package Management (NuGetAudit=true) + npm audit = 0 vulnerabilities.	Directory.Packages.props

SECTION 5

Step-by-Step Local Machine Setup & Execution Guide

Follow these step-by-step instructions to set up, build, run, and verify SecurePortal on your local machine.

Step 1: Verify Installed Prerequisites

```
dotnet --version   # Must be 10.0.100+ (e.g., 10.0.400)
node --version     # Must be v22.12+ or v24+ (e.g., v24.18.0)
npm --version      # Must be 10.0+
sqlcmd -?          # Verifies SQL Server CLI connectivity
```

Step 2: Navigate to the Workspace Directory

```
cd C:\Projects\secure-portal
```

Step 3: Database Location & Initialization (SecurePortalDb)

Database Physical Location & Connection String
The application connects to your local SQL Server instance on localhost (Port 1433) using Windows Authentication:
• **Connection String:** Server=localhost;Database=SecurePortalDb;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultSets=true;
• **Primary Data File (.mdf):** C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER\MSSQL\DATA\SecurePortalDb.mdf
• **Transaction Log (.ldf):** C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER\MSSQL\DATA\SecurePortalDb_log.ldf

On a brand-new machine, the database is automatically migrated and seeded when the API starts, or you can run:

```
# Option A: Apply EF Core 10 Migrations Directly
dotnet ef database update --project backend/src/Infrastructure --startup-project backend/src/Api

# Option B: Or launch SQL Server in Docker if local SQL Server is not installed
docker-compose up -d database
```

Step 4: Build & Audit Backend and Frontend

```
# 1. Compile .NET 10 Solution (Expected: 0 Warning(s), 0 Error(s))
dotnet build backend/SecurePortal.slnx

# 2. Audit Backend NuGet Packages (Expected: 0 vulnerable packages)
dotnet list backend/SecurePortal.slnx package --vulnerable --include-transitive

# 3. Install & Build Angular 22 Frontend (Expected: 0 errors, ~84.41 kB initial transfer size)
cd frontend
npm install
npm run build
npm audit
cd ..
```

Step 5: Launch the Full-Stack Environment

Option A — One-Command Concurrent Launcher (Recommended): Launches both .NET 10 API and Angular 22 in separate PowerShell windows:

```
powershell -ExecutionPolicy Bypass -File scripts/start-dev.ps1
```

Option B — Manual Two-Terminal Startup:

```
# Terminal 1 (Backend API on http://localhost:5047)
cd C:\Projects\secure-portal
dotnet run --project backend/src/Api

# Terminal 2 (Angular 22 SPA on http://localhost:4200)
cd C:\Projects\secure-portal\frontend
npm start
```

Step 6: Log In & Explore Application URLs

- **Frontend UI:** http://localhost:4200
- **Backend REST API:** http://localhost:5047/api
- **Swagger OpenAPI Docs:** http://localhost:5047/swagger
- **Default Administrator Credentials:** Username: admin (or admin@secureportal.local) | Password: Admin@Secure2026!P@ss

Step 7: Run the 22-Point Automated Security Verification Suite

```
powershell -ExecutionPolicy Bypass -File scripts/test-security-e2e.ps1
```

Automated Verification Result: 22 PASSED, 0 FAILED
Validates CSP headers, X-Frame-Options, nosniff, Server header removal, XSRF-TOKEN cookie issuance, FluentValidation HTTP 400 ProblemDetails, SQL Injection defense, Admin JWT login, Member registration, RBAC (403 vs 200 on /api/audit), sensitive field redaction ([REDACTED]), Keyset pagination, Rate Limiting (HTTP 429), and NuGet/NPM zero-vulnerability checks.

SECTION 6

Developer Troubleshooting & Lessons Learned FAQ

1. Why did running 'ng serve' previously load the Production environment file?

Root Cause: In Angular 22 (with the @angular/build:application builder), all services import from src/environments/environment. Without a fileReplacements rule under architect.build.configurations.development in frontend/angular.json, Angular bundles the base environment.ts (which has production: true and apiUrl: '/api') even during ng serve.

Fix Applied in frontend/angular.json:

```
"development": {
  "optimization": false,
  "extractLicenses": false,
  "sourceMap": true,
  "fileReplacements": [
    {
      "replace": "src/environments/environment.ts",
      "with": "src/environments/environment.development.ts"
    }
  ]
}
```

2. What causes the red toaster 'Unable to communicate with the secure server' on login?

Root Cause: Angular's error.interceptor.ts displays this message when an HTTP request returns **Status Code 0 (Network / CORS Error)**. This happens when:

- The .NET 10 backend API is not running on http://localhost:5047.
- A stale backend process is running from an older build before http://localhost:4200 was added to Cors:AllowedOrigins.
- UseHttpsRedirection() was placed before UseCors() in Development, redirecting HTTP preflight OPTIONS requests without CORS headers.

Fix Applied: app.UseCors("AllowAngularClient") is placed before UseHttpsRedirection() in Program.cs, UseHttpsRedirection() is guarded by if (!app.Environment.IsDevelopment()), and both http://localhost:4200 and https://localhost:4200 are whitelisted in appsettings.json and appsettings.Development.json.

3. How do I resolve build error MSB3027 / MSB3021 (File locked by SecurePortal.Api)?

Root Cause: An instance of SecurePortal.Api.exe is actively running and locking the output binary in bin\Debug\net10.0\.

Resolution: Stop the running backend terminal (Ctrl+C) or run the following PowerShell command before rebuilding:

```
Stop-Process -Name "SecurePortal.Api" -Force -ErrorAction SilentlyContinue
```

4. How are Angular components structured in this project?

All Angular 22 components follow a strict separation of concerns with three dedicated files per component: TypeScript class logic (.component.ts), HTML template (.component.html via templateUrl), and scoped CSS stylesheet (.component.css via styleUrl).